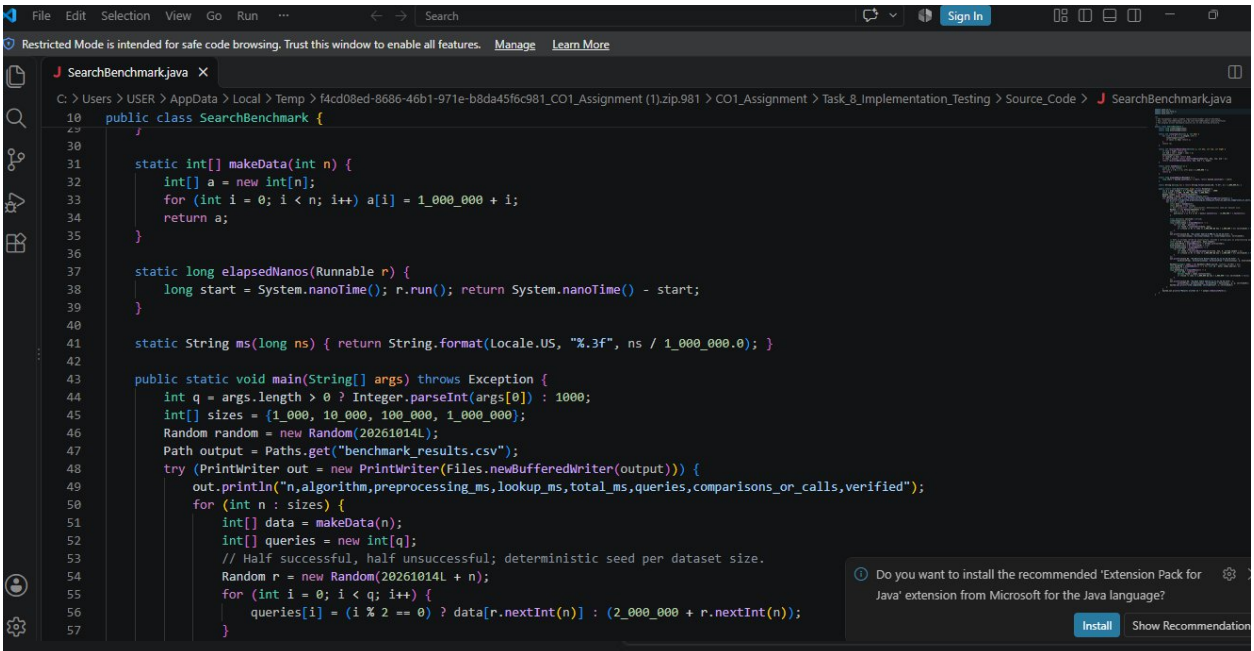


Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
SearchBenchmark.java
C: > Users > USER > AppData > Local > Temp > f4cd08ed-8686-46b1-971e-b8da45f6c981_CO1_Assignment (1).zip.981 > CO1_Assignment

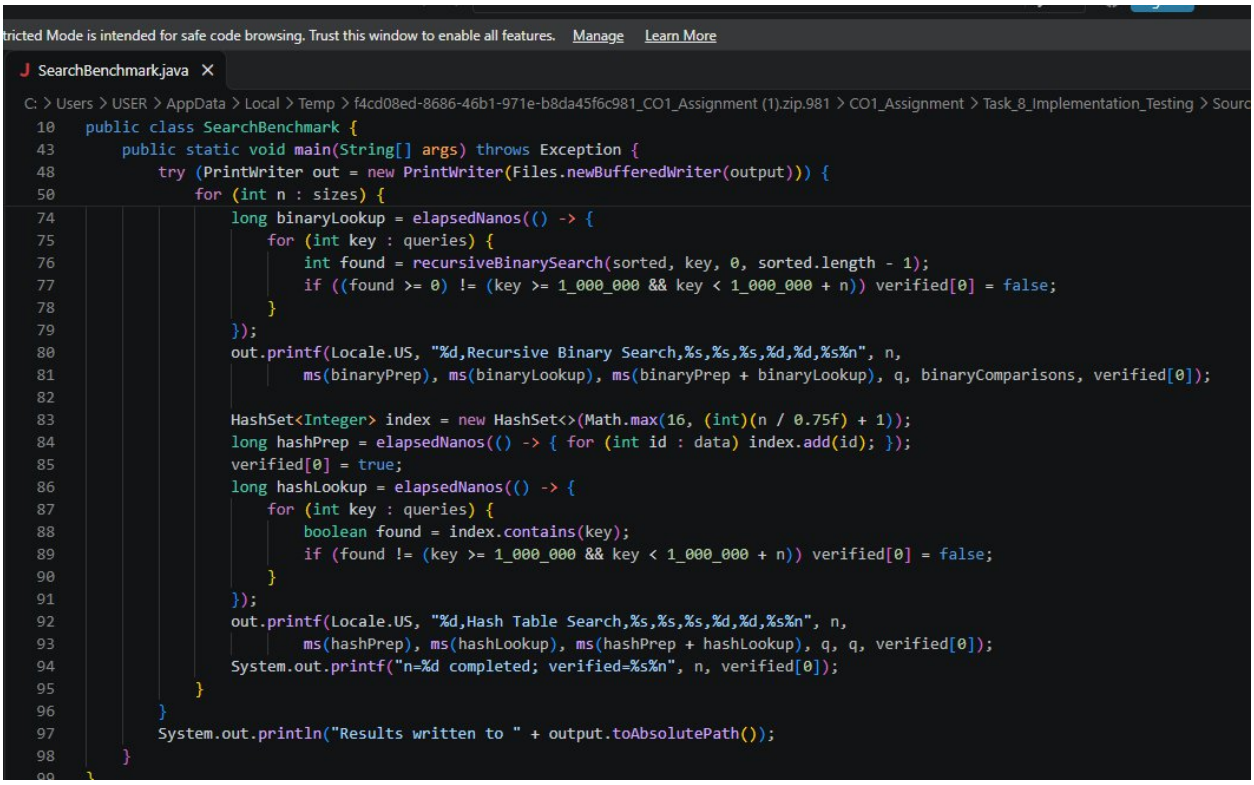
```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

/**
 * CO1 Assignment: exact student registration-number search benchmark.
 * Run: javac SearchBenchmark.java && java SearchBenchmark [queriesPerSize]
 * The program writes benchmark_results.csv in the working directory.
 */
public class SearchBenchmark {
    static long linearComparisons;
    static long binaryComparisons;

    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            linearComparisons++;
            if (a[i] == key) return i;
        }
        return -1;
    }

    static int recursiveBinarySearch(int[] a, int key, int low, int high) {
        if (low > high) return -1;
        int mid = low + (high - low) / 2;
        binaryComparisons++;
        if (a[mid] == key) return mid;
        if (key < a[mid]) return recursiveBinarySearch(a, key, low, mid - 1);
        return recursiveBinarySearch(a, key, mid + 1, high);
    }

```


tricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
SearchBenchmark.java
C: > Users > USER > AppData > Local > Temp > f4cd08ed-8686-46b1-971e-b8da45f6c981_CO1_Assignment (1).zip.981 > CO1_Assignment > Task_8_Implementation_Testing > Sourc
10 public class SearchBenchmark {
43 public static void main(String[] args) throws Exception {
48 try (PrintWriter out = new PrintWriter(Files.newBufferedWriter(output))) {
50 for (int n : sizes) {
74 long binaryLookup = elapsedNanos(() -> {
75 for (int key : queries) {
76 int found = recursiveBinarySearch(sorted, key, 0, sorted.length - 1);
77 if ((found >= 0) != (key >= 1_000_000 && key < 1_000_000 + n)) verified[0] = false;
78 }
79 });
80 out.printf(Locale.US, "%d,Recursive Binary Search,%s,%s,%s,%d,%d,%s%n", n,
81 ms(binaryPrep), ms(binaryLookup), ms(binaryPrep + binaryLookup), q, binaryComparisons, verified[0]);
82
83 HashSet<Integer> index = new HashSet<>(Math.max(16, (int)(n / 0.75f) + 1));
84 long hashPrep = elapsedNanos(() -> { for (int id : data) index.add(id); });
85 verified[0] = true;
86 long hashLookup = elapsedNanos(() -> {
87 for (int key : queries) {
88 boolean found = index.contains(key);
89 if (found != (key >= 1_000_000 && key < 1_000_000 + n)) verified[0] = false;
90 }
91 });
92 out.printf(Locale.US, "%d,Hash Table Search,%s,%s,%s,%d,%d,%s%n", n,
93 ms(hashPrep), ms(hashLookup), ms(hashPrep + hashLookup), q, q, verified[0]);
94 System.out.printf("n=%d completed; verified=%s%n", n, verified[0]);
95 }
96 }
97 System.out.println("Results written to " + output.toAbsolutePath());
98 }